DAST -- SECURITY -- AI-POWERED

ProbeX

Dynamic Application Security Testing Scanner

Find real vulnerabilities. Kill false positives. Get AI-powered fixes -- all on localhost.

7 Detector Classes | Real-time WebSocket Stream | Zero External Services | Gemini AI

Submitted by: **Chitransh Singh**

Project Proposal | Selection Round | 2026

01

Project Objective

ProbeX is a fully-functional, localhost-first **Dynamic Application Security Testing (DAST) scanner** built to detect real vulnerabilities in web applications -- without cloud dependencies, without SaaS subscriptions, and without false-positive noise. It is designed for developers and security engineers who need actionable, evidence-backed security feedback during development.

The tool achieves three concrete goals:

[1] **Automated vulnerability discovery** -- An async BFS crawler maps every endpoint, form, and parameter of the target application.

[2] **Baseline-differential false-positive defence** -- Mathematically proves a finding is real before reporting it -- eliminating whole classes of noise.

[3] **AI-powered remediation** -- Gemini generates context-aware, developer-friendly fix guidance beyond generic advice.

7	2	81	Gemini
Detector Classes	Scan Profiles	Tests Passing	AI Engine

02

Problem Statement

Web application vulnerabilities -- XSS, SQL Injection, misconfigured security headers -- remain the top attack vector in data breaches year after year (OWASP Top 10). Yet most developers ship code without ever running a security scan. Existing tools fail them in four ways:

[1] Cloud-only or subscription-gated

Burp Suite Pro costs $449/year. Most SAST/DAST SaaS platforms require sending your code or live traffic to external servers.

[2] Drowning in false positives

A scanner that fires on every reflected string is ignored within days. Engineers need verified, evidence-backed findings -- not a noise machine.

[3] Remediation without context

Generic advice like "use prepared statements" helps a senior engineer but leaves a junior developer without a clear next step.

[4] Complex to operate

Commercial scanners require multi-hour setup, proxy configuration, and deep security expertise just to run a first scan.

ProbeX solves all four problems in a single, self-contained, open-source tool that runs on localhost in under 30 seconds -- no account, no cloud, no configuration.

03

Key Features

[BFS] Async Crawler with Scope Lock

Concurrent breadth-first search discovers every endpoint, form, and query parameter. A hard scope lock ensures the crawler never follows links to external domains -- critical for safe use in local dev environments.

[FP] Baseline-Differential False Positive Defence

Every probe is compared against a clean baseline captured before injection. A pattern must appear NEW in the fuzzed response to count as a signal. A page that already says "syntax error" in its footer can never be wrongly flagged as SQL Injection.

[DET] Seven Vulnerability Detector Classes

Reflected XSS (DOM-verified), SQL Injection error-based (20 DB signatures), Missing Security Headers (CSP, X-Frame-Options, HSTS), Open Redirect, Path Traversal, SSTI, and SSRF -- each with CWE, OWASP Top 10, and CVSS score.

[DOM] DOM-Based XSS Verification

Rather than checking if <script> appears in response text (naive -- fires on escaped output), ProbeX parses the DOM with BeautifulSoup. The nonce must sit inside an executable script tag or event-handler attribute -- not a text node.

[WS] Real-Time WebSocket Progress Stream

Every status change, finding, and log line is broadcast over WebSocket as it happens. Late-joining clients receive full event replay so a page reload loses nothing.

[AI] Gemini AI Remediation Guidance

Each finding is enriched with Gemini-generated, context-specific fix guidance. The prompt includes the URL, parameter, payload, evidence, and CWE -- so the advice is tailored, not generic. An offline fallback handles the no-key scenario.

[RPT] Exportable Reports (JSON + HTML)

Every scan produces a machine-readable JSON report and a self-contained dark-theme HTML report with collapsible finding cards, severity badges, and a risk-score bar.

[DB] SQLite Persistence and Full Audit Log

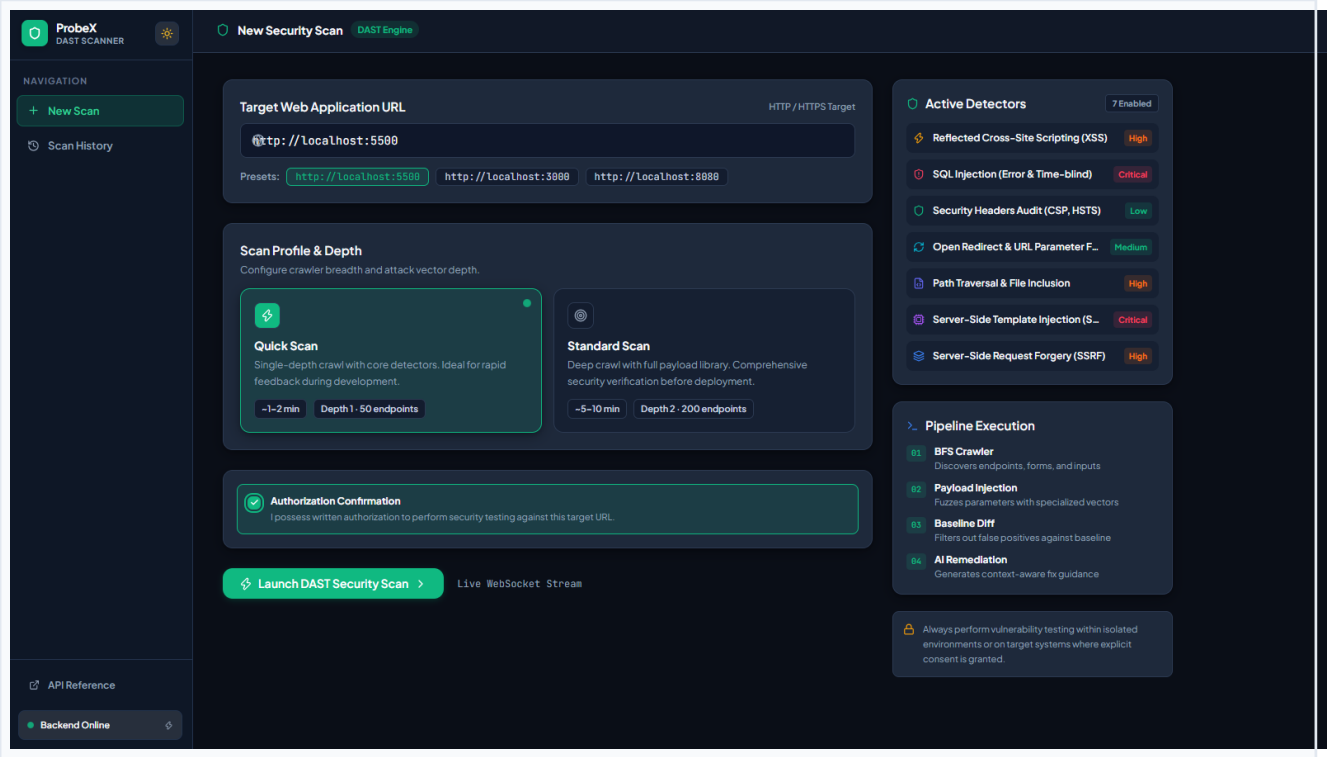
All scans and findings persist in local SQLite with timestamps, consent records, risk scores, and raw evidence. The Scan History view shows aggregate analytics across every run.

04

Project Screenshots

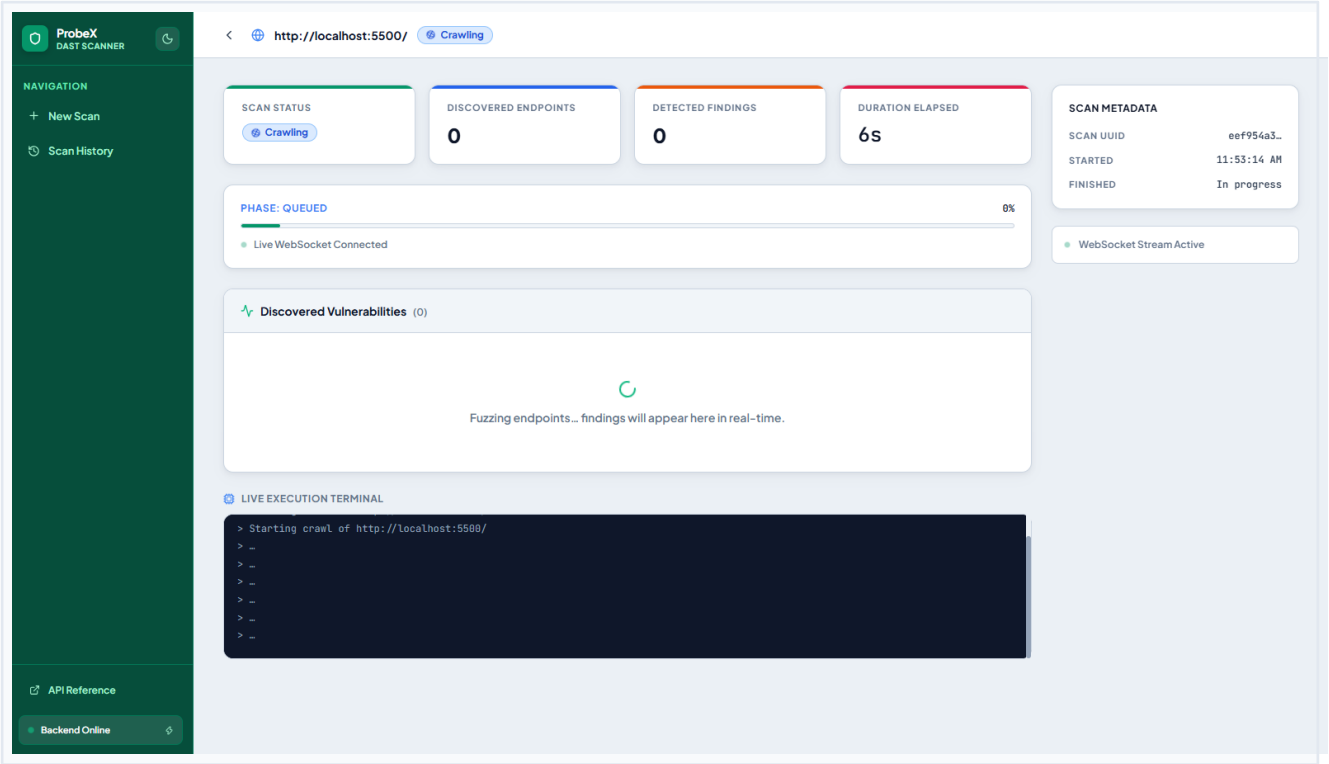
The following screenshots show ProbeX running live against a local vulnerable target. All interfaces are fully implemented -- this is not a mockup or design concept.

[01] Screen 1



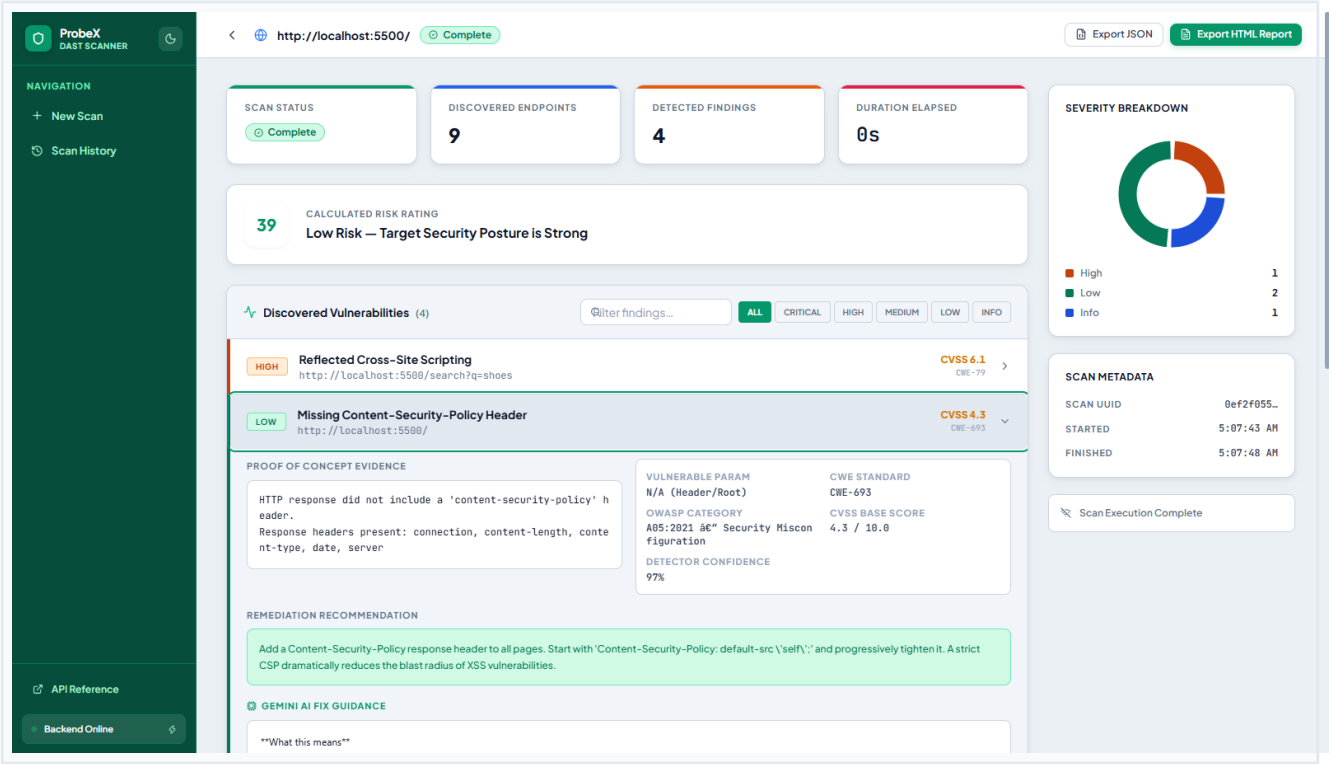
Screen 1 -- New Scan Interface: Target URL, scan profile selector, and active detector panel.

[02] Screen 2



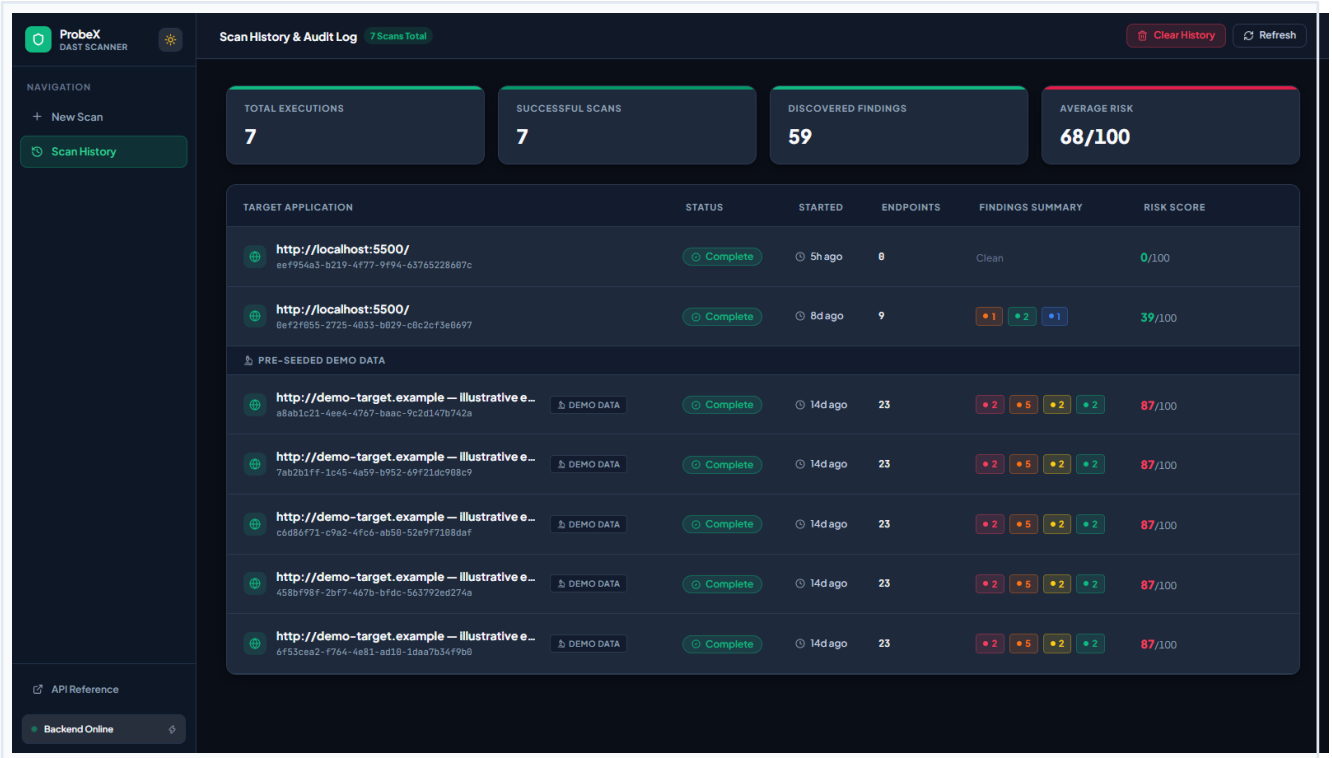
Screen 2 -- Live Scan Progress: Real-time WebSocket terminal, phase tracker, and live findings feed.

[03] Screen 3



Screen 3 -- Scan Results: Vulnerability cards with CVSS scores, CWE refs, evidence, and Gemini AI guidance.

[04] Screen 4



Screen 4 -- Scan History and Audit Log: Full audit trail with aggregate stats and severity breakdown.

Technology Stack

Layer	Technology	Purpose
Backend	FastAPI + Python 3.12	Async REST API and WebSocket server
Engine	httpx + BeautifulSoup4	HTTP client, BFS crawler, DOM parsing
Database	SQLite + SQLModel	Zero-config local persistence
Frontend	React + Vite + TypeScript	Real-time dashboard and report viewer
AI	Gemini API (google-generativeai)	Context-aware remediation guidance
Testing	pytest + respx	81 tests passing (100% green)

ProbeX is **fully implemented and running** -- backend, frontend, all detectors, AI integration, and 81 passing tests. It is ready for live demonstration on localhost with a single `python -m uvicorn app.main:app` command.